第11章 两个表的亲密接触——连接的原理

关系型数据库一个至关重要的概念就是 Join（连接）。相信很多同学在初学连接的时候一脸懵懂，理解了连接的语义之后又可能不明白各个表中的记录到底是怎么连起来的，以至于在使用的时候常常陷入下面这两种误区：

- 业务至上，不管三七二十一，再复杂的查询也在一个连接语句中搞定；
- 敬而远之，DBA 上次报过来的慢查询就是因为使用了连接导致的，以后再也不敢用了。

本章就来唠叨一下连接的原理。考虑到有些读者可能忘了连接是啥，或者压根儿就不知道连接是啥，为了节省他们宝贵的时间以及为了给咱们这本书凑字数，咱们先来介绍一下 MySQL 中支持的连接语法。

11.1 连接简介

11.1.1 连接的本质

为了故事的顺利发展，我们先建立两个简单的表，并给它们填充一点数据：

```
mysql> CREATE TABLE t1 (m1 int, n1 char(1));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE t2 (m2 int, n2 char(1));
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO t1 VALUES(1, 'a'), (2, 'b'), (3, 'c');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO t2 VALUES(2, 'b'), (3, 'c'), (4, 'd');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

我们成功建立了 t1、t2 两个表。这两个表都有两个列：一个是 INT 类型的；另外一个是 CHAR(1) 类型的。填充好数据的两个表看起来如下面这样：

```
mysql> SELECT * FROM t1;
+------+------+
| m1   | n1   |
+------+------+
|    1 | a    |
|    2 | b    |
|    3 | c    |
+------+------+
```

```
3 rows in set (0.00 sec)

mysql> SELECT * FROM t2;
+------+------+
| m2   | n2   |
+------+------+
|    2 | b    |
|    3 | c    |
|    4 | d    |
+------+------+
3 rows in set (0.00 sec)
```

从本质上来说，连接就是把各个表中的记录都取出来进行依次匹配，并把匹配后的组合发送给客户端。把 t1 和 t2 两个表连接起来的过程如图 11-1 所示。

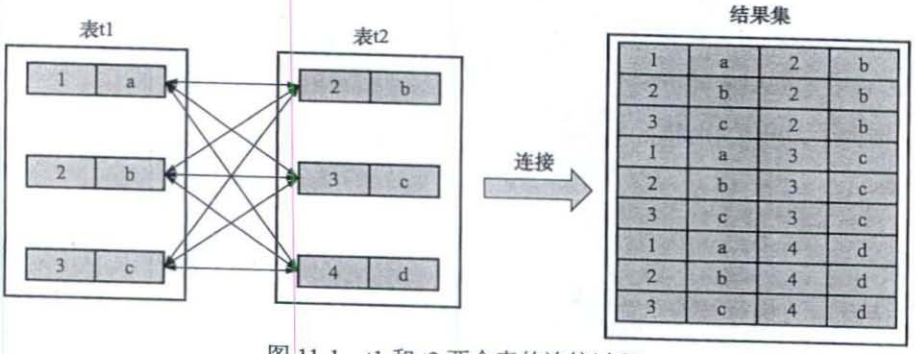

图 11-1　t1 和 t2 两个表的连接过程

这个过程看起来就是把 t1 表中的记录和 t2 表中的记录连起来组成一个新的更大的记录，所以这个查询过程称为连接查询。如果连接查询的结果集中包含一个表中的每一条记录与另一个表中的每一条记录相互匹配的组合，那么这样的结果集就可以称为笛卡儿积。因为表 t1 中有 3 条记录，表 t2 中也有 3 条记录，所以这两个表连接之后的笛卡儿积就有 $3 \times 3 = 9$ 条记录。在 MySQL 中，连接查询的语法也很随意，只要在 FROM 语句后边跟多个表名就好了。比如，我们把 t1 表和 t2 表连接起来的查询语句可以写成这样：

```
mysql> SELECT * FROM t1, t2;
+------+------+------+------+
| m1   | n1   | m2   | n2   |
+------+------+------+------+
|    1 | a    |    2 | b    |
|    2 | b    |    2 | b    |
|    3 | c    |    2 | b    |
|    1 | a    |    3 | c    |
|    2 | b    |    3 | c    |
|    3 | c    |    3 | c    |
|    1 | a    |    4 | d    |
|    2 | b    |    4 | d    |
|    3 | c    |    4 | d    |
+------+------+------+------+
9 rows in set (0.00 sec)
```

11.1.2 连接过程简介

如果乐意，我们可以连接任意数量的表。但是如果不附加任何限制条件，这些表连接起来产生的笛卡儿积可能是非常巨大的。比如，3 个 100 行记录的表连接起来产生的笛卡儿积就有 100 × 100 × 100 = 1,000,000 行记录！所以在连接时过滤掉特定的记录组合是有必要的。在连接查询中的过滤条件可以分成下面两种。

- 涉及单表的条件：这种只涉及单表的过滤条件已经在前文中提到过一万遍了，我们之前也一直称为搜索条件。比如 t1.m1>1 是只针对 t1 表的过滤条件，t2.n2<'d' 是只针对 t2 表的过滤条件。
- 涉及两表的条件：这种过滤条件我们之前没见过，比如 t1.m1=t2.m2、t1.n1>t2.n2 等。这些条件涉及了两个表，稍后会详细分析这种过滤条件是如何使用的。

下边我们看一下携带过滤条件的连接查询的大致执行过程，比如说下面这个查询语句：

```
SELECT * FROM t1, t2 WHERE t1.m1 > 1 AND t1.m1 = t2.m2 AND t2.n2 < 'd';
```

在这个查询中，我们指明了 3 个过滤条件：

- t1.m1>1；
- t1.m1=t2.m2；
- t2.n2<'d'。

这个连接查询的执行过程大致如下。

步骤 1. 首先确定第一个需要查询的表，这个表称为驱动表。

怎样在单表中执行查询语句已经在前一章中都唠叨过了：只需要选取代价最小的那种访问方法去执行单表查询语句就好了（就是说从 const、ref、ref_or_null、range、index merge、index、all 这些执行方法中选取代价最小的去执行查询即可）。这里假设使用 t1 作为驱动表，那么就需要到 t1 表中查找满足 t1.m1>1 的记录。因为表中的数据太少，我们也没在表上建立二级索引，所以我们将查询 t1 表所用的访问方法设定为 all，也就是采用全表扫描的方式执行单表查询。查询过程如图 11-2 所示。

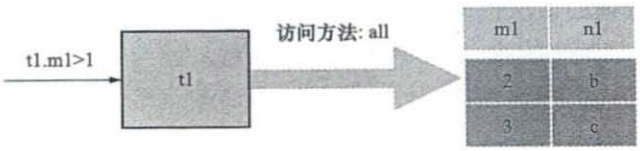

图 11-2 查询过程

可以看到，t1 表中符合 t1.m1>1 的记录有 2 条。

步骤 2. 步骤 1 中从驱动表每获取到一条记录，都需要到 t2 表中查找匹配的记录。

所谓匹配的记录，指的是符合过滤条件的记录。因为是根据 t1 表中的记录去找 t2 表中的记录，所以 t2 表也可以称为被驱动表。步骤 1 从驱动表中得到了 2 条记录，也就意味着需要查询 2 次 t2 表。此时涉及两个表的列的过滤条件 t1.m1 = t2.m2 就派上用场了。

对于从 t1 表中查询得到的第一条记录，也就是当 t1.m1=2 时，过滤条件 t1.m1=t2.m2 就相当于 t2.m2=2。所以此时 t2 表相当于有了 t2.m2=2、t2.n2<'d' 这两个过滤条件，然后到 t2 表中执行单表查询。

对于从 t1 表中查询得到的第二条记录，也就是当 t1.m1=3 时，过滤条件 t1.m1=t2.m2 就相当于 t2.m2=3。所以此时 t2 表相当于有了 t2.m2=3、t2.n2<'d' 这两个过滤条件，然后到 t2 表中

执行单表查询。

所以整个连接查询的执行过程如图 11-3 所示。

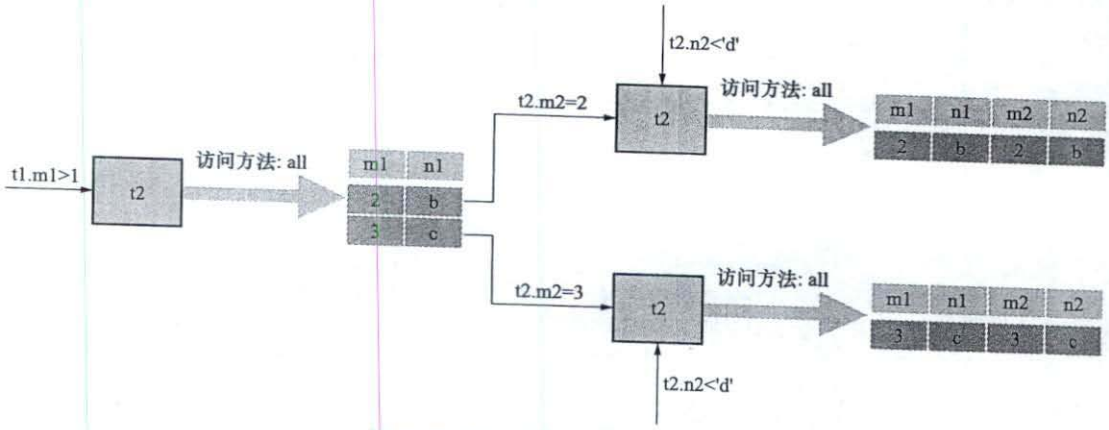

图 11-3 整个连接查询的执行过程

也就是说，整个连接查询最后的结果只有 2 条符合过滤条件的记录：

```
+------+------+------+------+
| m1   | n1   | m2   | n2   |
+------+------+------+------+
|    2 | b    |    2 | b    |
|    3 | c    |    3 | c    |
+------+------+------+------+
```

从上面的两个步骤可以看出，我们上边唠叨的这个两表连接查询共需要查询 1 次 t1 表、2 次 t2 表。当然这是特定过滤条件下的结果。如果把 t1.m1>1 这个条件去掉，那么从 t1 表中查出的记录就有 3 条，就需要查询 3 次 t2 表了。也就是说，在两表的连接查询中，驱动表只需要访问一次，被驱动表可能需要访问多次。

小贴士

这里需要强调一下，并不是将所有满足条件的驱动表记录先查询出来放到一个地方，然后再去被驱动表中查询的（大家可以试想一下，如果满足条件的驱动表记录非常多，那得需要多大一片存储空间呀），而是每获得一条驱动表记录，就立即到被驱动表中寻找匹配的记录。

11.1.3 内连接和外连接

为了大家更好地理解后面的内容，我们先创建两个有现实意义的表，

```
CREATE TABLE student (
    number INT NOT NULL AUTO_INCREMENT COMMENT '学号',
    name VARCHAR(5) COMMENT '姓名',
    major VARCHAR(30) COMMENT '专业',
    PRIMARY KEY (number)
) Engine=InnoDB CHARSET=utf8 COMMENT '学生信息表';

CREATE TABLE score (
    number INT COMMENT '学号',
```

```
    subject VARCHAR(30) COMMENT '科目',
    score TINYINT COMMENT '成绩',
    PRIMARY KEY (number, subject)
) Engine=InnoDB CHARSET=utf8 COMMENT '学生成绩表';
```

这里新建了一个学生信息表和一个学生成绩表。我们向这两个表中插入一些数据。为了节省篇幅，具体插入过程就不唠叨了。插入数据后的表如下所示：

```
mysql> SELECT * FROM student;
+----------+-----------+-------------------------+
| number   | name      | major                   |
+----------+-----------+-------------------------+
| 20180101 | 张三      | 软件学院                |
| 20180102 | 李四      | 计算机科学与工程        |
| 20180103 | 王五      | 计算机科学与工程        |
+----------+-----------+-------------------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM score;
+----------+----------------------------+-------+
| number   | subject                    | score |
+----------+----------------------------+-------+
| 20180101 | MySQL是怎样运行的          |    78 |
| 20180101 | 深入浅出MySQL              |    88 |
| 20180102 | 深入浅出MySQL              |    98 |
| 20180102 | MySQL是怎样运行的          |   100 |
+----------+----------------------------+-------+
4 rows in set (0.00 sec)
```

现在，要想把各位学生的考试成绩都查询出来，就需要进行两表连接了（因为 score 表中没有姓名信息，所以不能单纯只查询 score 表）。连接过程就是从 student 表中取出记录，然后在 score 表中查找 number 相同的成绩记录，所以过滤条件就是 student.number=socre.number，整个查询语句就是下面这样：

```
mysql> SELECT * FROM student, score WHERE student.number = score.number;
+----------+-----------+-----------------------+----------+----------------------------+-------+
| number   | name      | major                 | number   | subject                    | score |
+----------+-----------+-----------------------+----------+----------------------------+-------+
| 20180101 | 张三      | 软件学院              | 20180101 | MySQL是怎样运行的          |    78 |
| 20180101 | 张三      | 软件学院              | 20180101 | 深入浅出MySQL              |    88 |
| 20180102 | 李四      | 计算机科学与工程      | 20180102 | 深入浅出MySQL              |    98 |
| 20180102 | 李四      | 计算机科学与工程      | 20180102 | MySQL是怎样运行的          |   100 |
+----------+-----------+-----------------------+----------+----------------------------+-------+
4 rows in set (0.00 sec)
```

字段有点多，我们少查询几个字段：

```
mysql> SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1, score AS s2 WHERE
s1.number = s2.number;
+----------+-----------+----------------------------+-------+
| number   | name      | subject                    | score |
+----------+-----------+----------------------------+-------+
| 20180101 | 张三      | MySQL是怎样运行的          |    78 |
| 20180101 | 张三      | 深入浅出MySQL              |    88 |
```

```
| 20180102 | 李四        | 深入浅出MySQL                  |    98 |
| 20180102 | 李四        | MySQL是怎样运行的              |   100 |
+----------+-----------+-----------------------------+-------+
4 rows in set (0.00 sec)
```

从上述查询结果中可以看到，各位学生对应的各科成绩都被查出来了。可是有个问题：王五同学（也就是学号为 20180103 的同学）因为某些原因没有参加考试，所以在 score 表中没有对应的成绩记录。如果老师想查看所有学生的考试成绩，即使是缺考的学生，他们的成绩也应该展示出来，但是到目前为止我们介绍的连接查询无法完成这样的需求。我们稍微思考一下这个需求，其本质是这样的：针对驱动表中的某条记录，即使在被驱动表中没有找到与之匹配的记录，也仍然需要把该驱动表记录加入到结果集。为了解决这个问题，就有了内连接和外连接的概念。

- 对于内连接的两个表，若驱动表中的记录在被驱动表中找不到匹配的记录，则该记录不会加入到最后的结果集。前面提到的连接都是内连接。
- 对于外连接的两个表，即使驱动表中的记录在被驱动表中没有匹配的记录，也仍然需要加入到结果集。

在 MySQL 中，根据选取的驱动表的不同，外连接可以细分为 2 种。

- 左外连接：选取左侧的表为驱动表。
- 右外连接：选取右侧的表为驱动表。

可这样仍然存在问题：即使对于外连接来说，有时候我们也不想把驱动表的全部记录都加入到最后的结果集。这就犯难了，有时候匹配失败要加入结果集，有时候又不要加入结果集。这咋办，有点儿犯难啊。把过滤条件分为两种不就解决这个问题了么，所以过滤条件在不同的地方是有不同的语义的。

- WHERE 子句中的过滤条件

WHERE 子句中的过滤条件就是我们平时见的那种。不论是内连接还是外连接，凡是不符合 WHERE 子句中过滤条件的记录都不会被加入到最后的结果集。

- ON 子句中的过滤条件

对于外连接的驱动表中的记录来说，如果无法在被驱动表中找到匹配 ON 子句中过滤条件的记录，那么该驱动表记录仍然会被加入到结果集中，对应的被驱动表记录的各个字段使用 NULL 值填充。

需要注意的是，这个 ON 子句是专门为“外连接驱动表中的记录在被驱动表找不到匹配记录时是否应该把该驱动表记录加入结果集中”这个场景提出的。所以，如果把 ON 子句放到内连接中，MySQL 会把它像 WHERE 子句一样对待。也就是说，内连接中的 WHERE 子句和 ON 子句是等价的。

小贴士

左外连接和右外连接分别简称为左连接和右连接，所以下面提到的左外连接和右外连接中的“外”字都用括号扩起来，表示这个字儿可有可无。

1. 左（外）连接的语法

左（外）连接的语法还是挺简单的。比如，我们要把 t1 表和 t2 表进行左外连接查询，可以这么写：

```
SELECT * FROM t1 LEFT [OUTER] JOIN t2 ON 连接条件 [WHERE 普通过滤条件];
```

其中，中括号里的 OUTER 单词是可以省略的。对于 LEFT JOIN 类型的连接来说，我们把放在左边的表称为外表或者驱动表，放在右边的表称为内表或者被驱动表。所以在上述查询语句中，t1 就是外表或者驱动表，t2 就是内表或者被驱动表。需要注意的是，对于左（外）连接和右（外）连接来说，必须使用 ON 子句来指出连接条件（内连接不必要包含 ON 子句）。了解了左（外）连接的基本语法之后，再次回到前面的那个现实问题中来，看看怎样写查询语句才能把所有学生（即使是缺考的学生）的成绩信息都查询出来，并放到结果集中：

```
mysql> SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1 LEFT JOIN score AS
s2 ON s1.number = s2.number;
+----------+-----------+----------------------------+-------+
| number   | name      | subject                    | score |
+----------+-----------+----------------------------+-------+
| 20180101 | 张三      | MySQL是怎样运行的          |    78 |
| 20180101 | 张三      | 深入浅出MySQL              |    88 |
| 20180102 | 李四      | 深入浅出MySQL              |    98 |
| 20180102 | 李四      | MySQL是怎样运行的          |   100 |
| 20180103 | 王五      | NULL                       |  NULL |
+----------+-----------+----------------------------+-------+
5 rows in set (0.04 sec)
```

从结果集中可以看出，虽然王五并没有对应的成绩记录，但是由于采用的连接类型为左（外）连接，所以仍然把他放到了结果集中，只不过在对应的成绩记录的各列使用了 NULL 值进行填充。

2. 右（外）连接的语法

右（外）连接和左（外）连接的原理是一样的，语法也只是把 LEFT 换成 RIGHT 而已：

```
SELECT * FROM t1 RIGHT [OUTER] JOIN t2 ON 连接条件 [WHERE 普通过滤条件];
```

在右（外）连接中，只不过驱动表是右边的表，被驱动表是左边的表，这里不再赘述。

3. 内连接的语法

内连接和外连接的根本区别就是在驱动表中的记录不符合 ON 子句中的连接条件时，内连接不会把该记录加入到最后的结果集中。我们最开始唠叨的那些连接查询的类型都是内连接。不过之前仅提到了一种最简单的内连接语法，就是直接把需要连接的多个表都放到 FROM 子句后面。其实 MySQL 针对内连接提供了好多不同的语法，我们以 t1 和 t2 表为例来看看：

```
SELECT * FROM t1 [INNER | CROSS] JOIN t2 [ON 连接条件] [WHERE 普通过滤条件];
```

也就是说在 MySQL 中，下面这几种内连接的写法都是等价的：

- SELECT * FROM t1 JOIN t2;
- SELECT * FROM t1 INNER JOIN t2;
- SELECT * FROM t1 CROSS JOIN t2;

上面这些写法等价于直接把需要连接的表名放到 FROM 语句之后，再用逗号分隔开的写法：

```
SELECT * FROM t1, t2;
```

尽管我们介绍了很多种内连接的书写方式，不过熟悉其中一种就好了，我个人比较推荐以 INNER JOIN 的形式书写内连接（因为 INNER JOIN 的语义很明确，可以与 LEFT JOIN 和

RIGHT JOIN 轻松地区分开）。这里需要注意的是，由于在内连接中 ON 子句和 WHERE 子句是等价的，所以内连接中不要求强制写明 ON 子句。

我们前面说过，连接就是把各个表中的记录都取出来依次进行匹配，并把匹配后的组合发送给客户端。无论哪个表作为驱动表，两表连接产生的笛卡儿积肯定是一样的。而对于内连接来说，凡是不符合 ON 子句或 WHERE 子句中条件的记录都会被过滤掉，也就相当于从两表连接的笛卡儿积中把不符合过滤条件的记录给踢出去（这里只是打个比方，并不是在真正执行查询时先获取笛卡儿尔积）。所以对于内连接来说，驱动表和被驱动表是可以互换的，并不会影响最后的查询结果。但是对于外连接来说，由于驱动表中的记录即使在被驱动表中找不到符合 ON 子句连接条件的记录，也会被加入到结果集，此时驱动表和被驱动表的关系就很重要了。也就是说，左外连接和右外连接的驱动表和被驱动表不能轻易互换。

4. 小结

上面说了这么多，给大家的感觉不是很直观，我们直接把表 t1 和 t2 的 3 种连接方式写在一起，这样大家理解起来就很容易了：

```
mysql> SELECT * FROM t1 INNER JOIN t2 ON t1.m1 = t2.m2;
+------+------+------+------+
| m1   | n1   | m2   | n2   |
+------+------+------+------+
|    2 | b    |    2 | b    |
|    3 | c    |    3 | c    |
+------+------+------+------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM t1 LEFT JOIN t2 ON t1.m1 = t2.m2;
+------+------+------+------+
| m1   | n1   | m2   | n2   |
+------+------+------+------+
|    2 | b    |    2 | b    |
|    3 | c    |    3 | c    |
|    1 | a    | NULL | NULL |
+------+------+------+------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM t1 RIGHT JOIN t2 ON t1.m1 = t2.m2;
+------+------+------+------+
| m1   | n1   | m2   | n2   |
+------+------+------+------+
|    2 | b    |    2 | b    |
|    3 | c    |    3 | c    |
| NULL | NULL |    4 | d    |
+------+------+------+------+
3 rows in set (0.00 sec)
```

11.2 连接的原理

上文那些啰嗦的介绍都只是为了唤醒大家对连接、内连接、外连接这些概念的回忆。这些

基本概念是为了真正进入本章主题所做的铺垫。真正的重点是 MySQL 采用了什么样的算法来进行表与表之间的连接。了解了这个之后，大家才能明白为啥有的连接查询的运行速度快如闪电，有的却慢如蜗牛。

11.2.1　嵌套循环连接

前文说过，对于两表连接来说，驱动表只会被访问一遍，但被驱动表却要被访问好多遍；具体访问几遍取决于对驱动表执行单表查询后的结果集中有多少条记录。对于内连接来说，选取哪个表为驱动表都没关系；而外连接的驱动表是固定的，也就是说左（外）连接的驱动表就是左边的那个表，右（外）连接的驱动表就是右边的那个表。前文已经介绍过 t1 表和 t2 表执行内连接查询的大致过程，我们现在温习一下。

步骤 1. 选取驱动表，使用与驱动表相关的过滤条件，选取代价最低的单表访问方法来执行对驱动表的单表查询。

步骤 2. 对步骤 1 中查询驱动表得到的结果集中的每一条记录，都分别到被驱动表中查找匹配的记录。

通用的两表连接过程如图 11-4 所示。

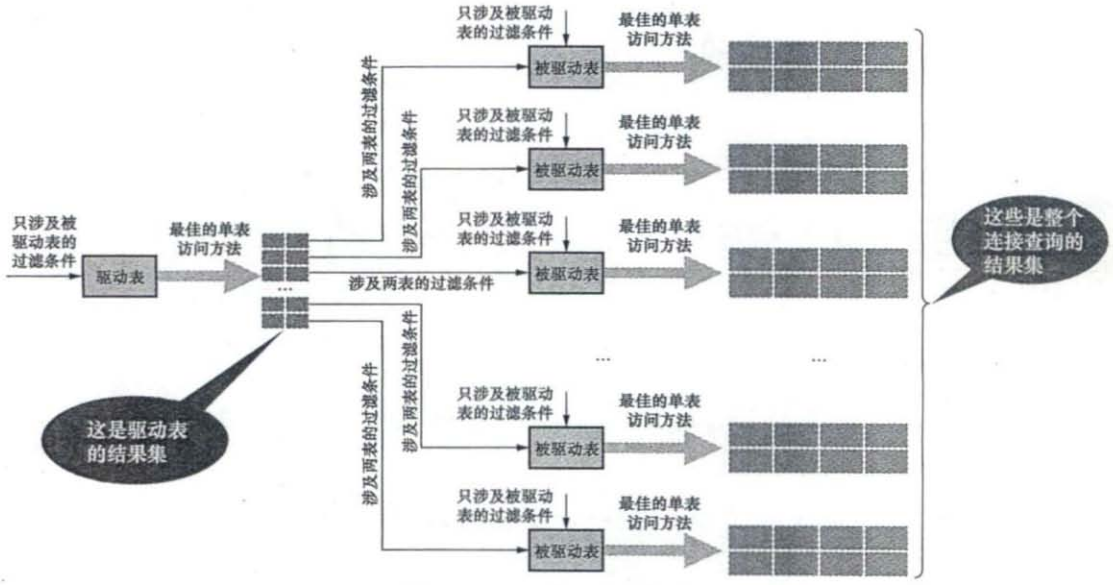

图 11-4　通用的两表连接过程

如果有 3 个表进行连接，那么步骤 2 中得到的结果集就像是新的驱动表，然后第 3 个表就成为了被驱动表，然后重复上面的过程。也就是针对步骤 2 中得到的结果集中的每一条记录，都需要到 t3 表中找一找有没有匹配的记录。用伪代码来表示这个过程就是下面这样：

```
for each row in t1 satisfying conditions about t1 {
    for each row in t2 satisfying conditions about t2 {
        for each row in t3 satisfying conditions about t3 {
            send to client;
```

```
        }
    }
}
```

这个过程就像是一个嵌套的循环，所以这种“驱动表只访问一次，但被驱动表却可能访问多次，且访问次数取决于对驱动表执行单表查询后的结果集中有多少条记录”的连接执行方式称为嵌套循环连接（Nested-Loop Join），这是最简单也是最笨拙的一种连接查询算法。

小贴士

需要注意的是，对于嵌套循环连接算法来说，每当我们从驱动表中得到了一条记录时，就根据这条记录立即到被驱动表中查询一次。如果得到了匹配的记录，就把组合后的记录发送给客户端，然后再到驱动表中获取下一条记录；这个过程将重复进行。我们在图 11-4 中提到的“结果集”只是一个抽象的概念，大家不要错误地以为是把驱动表中所有的记录都先查出来放到某个地方（比如内存或者磁盘中），然后再遍历这些记录。

11.2.2 使用索引加快连接速度

我们知道，在嵌套循环连接中可能需要访问多次被驱动表。如果访问被驱动表的方式都是全表扫描，那得要扫描好多次！但是别忘了，查询 t2 表其实就相当于一次单表查询，我们可以利用索引来加快查询速度。回顾最开始介绍的使用 t1 表和 t2 表进行内连接的例子：

```
SELECT * FROM t1, t2 WHERE t1.m1 > 1 AND t1.m1 = t2.m2 AND t2.n2 < 'd';
```

这个连接查询使用的是其实是嵌套循环连接算法。把上面这个查询执行过程拿出来给大家看一下，如图 11-5 所示。

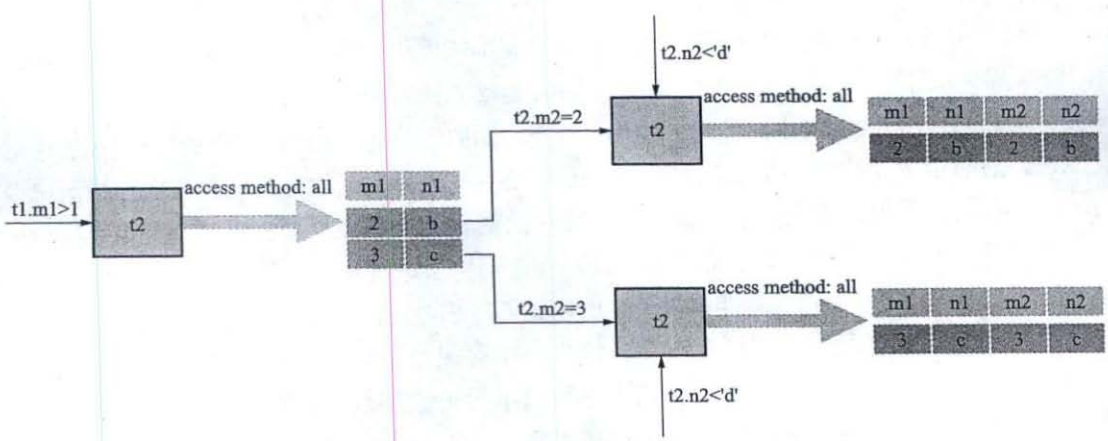

图 11-5 连接查询

查询驱动表 t1 后的结果集中有 2 条记录，嵌套循环连接算法需要查询被驱动表 2 次：

- 当 t1.m1=2 时，查询一遍 t2 表，对 t2 表的查询语句相当于：

```
SELECT * FROM t2 WHERE t2.m2 = 2 AND t2.n2 < 'd';
```

- 当 t1.m1=3 时，再去查询一遍 t2 表，此时对 t2 表的查询语句相当于：

```
SELECT * FROM t2 WHERE t2.m2 = 3 AND t2.n2 < 'd';
```

可以看到，原来的 t1.m1=t2.m2 这个涉及两个表的过滤条件在针对 t2 表进行查询时，关于 t1 表的条件就已经确定了，所以我们只需要单单优化针对 t2 表的查询即可。上述两个对 t2 表的查询语句中利用到的是 m2 和 n2 列，我们可以进行如下尝试。

- 在 m2 列上建立索引。因为针对 m2 列的条件是等值查找，比如 t2.m2=2、t2.m2=3 等，所以可能使用到 ref 访问方法。假设使用 ref 访问方法来执行对 t2 表的查询，需要在回表之后再判断 t2.n2<d 这个条件是否成立。

这里有一个比较特殊的情况，即假设 m2 列是 t2 表的主键，或者是不允许存储 NULL 值的唯一二级索引列，那么使用“t2.m2 = 常数值”这样的条件从 t2 表中查找记录时，代价就是常数级别的。我们知道，在单表中使用主键值或者唯一二级索引列的值进行等值查找的方式称为 const，而在连接查询中对被驱动表的主键或者不允许存储 NULL 值的唯一二级索引进行等值查找使用的访问方法就称为 eq_ref。

- 在 n2 列上建立索引，涉及的条件是 t2.n2 < 'd'，可能用到 range 访问方法。假设使用 range 访问方法对 t2 表进行查询，需要在回表之后再判断包含 m2 列的条件是否成立。

假设 m2 和 n2 列上都存在索引，那么就需要从这两个里面挑一个代价更低的索引来查询 t2 表。

另外，连接查询的查询列表和过滤条件中有时可能只涉及被驱动表的部分列，而这些列都是某个二级索引的一部分，在这种情况下即使不能使用 eq_ref、ref、ref_or_null 或者 range 等访问方法来查询被驱动表，也可以通过扫描全部二级索引记录（即使用 index 访问方法）来查询被驱动表。所以建议最好不要使用 * 作为查询列表，而是把真正用到的列作为查询列表。

11.2.3 基于块的嵌套循环连接

现实生活中的表可不像 t1、t2 这样只有 3 条记录，成千上万条记录都是少的，几百万、几千万甚至几亿条记录的表到处都是。现在假设我们不能使用索引加快被驱动表的查询过程，所以对于驱动表结果集中的每一条记录，都需要对被驱动表执行全表扫描。这样在对被驱动表进行全表扫描时，可能表前面的记录还在内存中，而表后面的记录还在磁盘上。而等到扫描表中后面的记录时，有可能由于内存不足，需要把表前面的记录从内存中释放掉给现在正在扫描的记录腾地方。我们前面强调过，在采用嵌套循环连接算法的两表连接过程中，被驱动表可是要被访问好多次。如果这个被驱动表中的数据特别多而且不能使用索引进行访问，那就相当于要从磁盘上读这个表好多次，这个 I/O 的代价就非常大了。所以我们得想办法，尽量减少被驱动表的访问次数。

通过上面的叙述我们了解到，驱动表结果集中有多少条记录，就可能把被驱动表从磁盘加载到内存中多少次。我们是否可以在把被驱动表中的记录加载到内存时，一次性地与驱动表中的多条记录进行匹配呢？这样就可以大大减少重复从磁盘上加载被驱动表的代价了。所以设计 MySQL 的大叔提出了一个名为 Join Buffer（连接缓冲区）的概念。Join Buffer 就是在执行连接查询前申请的一块固定大小的内存。先把若干条驱动表结果集中的记录装在这个 Join Buffer 中，然后开始扫描被驱动表，每一条被驱动表的记录一次性地与 Join Buffer 中的多条驱动表记录进行匹配。由于匹配的过程都是在内存中完成的，所以这样可以显著减少被驱动表的 I/O 代价。使用 Join Buffer 的过程如图 11-6 所示。

最好的情况是 Join Buffer 足够大，能容纳驱动表结果集中的所有记录，这样只需要访问一

次被驱动表就可以完成连接操作了。设计 MySQL 的大叔把这种加入了 Join Buffer 的嵌套循环连接算法称为基于块的嵌套循环连接（Block Nested-Loop Join）算法。

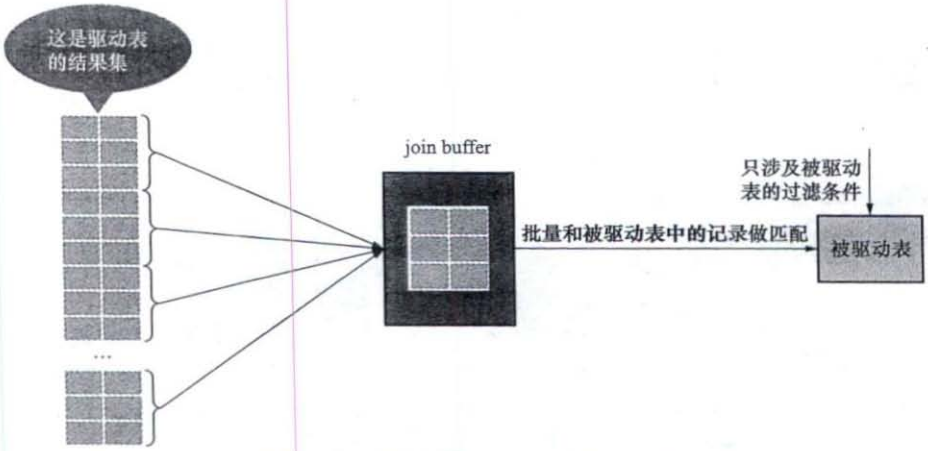

图 11-6 使用 Join Buffer 的过程示意图

这个 Join Buffer 的大小可以通过启动选项或者系统变量 join_buffer_size 进行配置，默认大小为 262,144 字节（也就是 256KB），最小可以设置为 128 字节。当然，在我们优化对被驱动表的查询时，最好是为被驱动表加上高效率的索引。如果实在不能使用索引，并且自己机器的内存也比较大，则可以尝试调大 join_buffer_size 的值来对连接查询进行优化。

另外需要注意的是，Join Buffer 中并不会存放驱动表记录的所有列，只有查询列表中的列和过滤条件中的列才会被放到 Join Buffer 中，所以这也再次提醒我们，最好不要把 * 作为查询列表，只需要把关心的列放到查询列表就好了；这样还可以在 Join Buffer 中放置更多的记录。

11.3 总结

从本质上来说，连接就是把各个表中的记录都取出来依次进行匹配，并把匹配后的组合发送给客户端。如果不加任何过滤条件，产生的结果集就是笛卡儿积。

在 MySQL 中，连接分为内连接和外连接，其中外连接又可以被细分为左（外）连接和右（外）连接。内连接和外连接的根本区别就是，在驱动表中的记录不符合 ON 子句中的连接条件时，内连接不会把该记录加入到最后的结果集中，而外连接会。

嵌套循环连接算法是指驱动表只访问一次，但被驱动表却可能会访问多次，访问次数取决于对驱动表执行单表查询后的结果集中有多少条记录。大致过程如下。

步骤 1. 选取驱动表，使用与驱动表相关的过滤条件，选取代价最低的单表访问方法来执行对驱动表的单表查询。

步骤 2. 对步骤 1 中查询驱动表得到的结果集中的每一条记录，都分别到被驱动表中查找匹配的记录。

由于被驱动表可能会访问多次，因此可以为被驱动表建立合适的索引以加快查询速度。

如果被驱动表非常大，多次访问被驱动表可能导致很多次的磁盘 I/O，此时可以使用基于块的嵌套循环连接算法来缓解由此造成的性能损耗。